

FACULDADE COTEMIG

FUNDAMENTOS TEÓRICOS DA COMPUTAÇÃO

**RELATÓRIO TÉCNICO: SIMULADOR DE MÁQUINAS
ABSTRATAS**

Membros da Equipe:

Alex Paulo Vianna Da Costa Candido e Silva

Carlos Henrique Queiroz Teixeira Ramos

Bernardo Medeiros Mendes

Belo Horizonte / MG

Junho de 2026

1 Introdução

Este relatório apresenta o desenvolvimento de um simulador computacional de máquinas abstratas, desenvolvido como requisito para a avaliação final da disciplina de Fundamentos Teóricos da Computação. O objetivo central do nosso trabalho foi trazer os modelos matemáticos teóricos — Autômatos Finitos Determinísticos (AFD), Autômatos de Pilha (AP) e Máquinas de Turing (MT) — para o ambiente prático, transformando-os em algoritmos executáveis na linguagem C#.

O estudo de linguagens formais nos dá a base necessária para entender os limites da computação. Com este projeto, nossa intenção foi validar conceitos fundamentais como o determinismo, o uso de memória restrita por pilhas e o comportamento de uma memória irrestrita por meio de uma fita infinita. Estruturamos o sistema de forma modular, aplicando validações baseadas nas tuplas formais de cada máquina. Isso permitiu ao simulador não apenas reconhecer as linguagens propostas, mas também resolver problemas de decisão e executar funções matemáticas reais, como a adição unária.

2 Fundamentação Teórica

2.1 Autômato Finito Determinístico (AFD)

Um AFD é um modelo matemático restrito que reconhece Linguagens Regulares. Ele é definido formalmente por uma 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$, onde: Q representa o conjunto finito de estados; Σ é o alfabeto de símbolos de entrada permitidos; $\delta : Q \times \Sigma \rightarrow Q$ dita a função de transição; $q_0 \in Q$ é o estado por onde a computação começa; e $F \subseteq Q$ é o conjunto de estados de aceitação. **Exemplo de Cálculo (L_1 - Termina em ab):** Ao processar a palavra $w = aab$, o simulador parte de q_0 . Ao ler o primeiro 'a', transita para q_1 . Lê o segundo 'a' e permanece em q_1 . Por fim, lê o símbolo 'b' e chega ao estado q_2 . Como $q_2 \in F$, a palavra é aceita.

2.2 Autômato de Pilha (AP)

Um AP é voltado para o reconhecimento de Linguagens Livres de Contexto, utilizando uma memória auxiliar que funciona no padrão LIFO (tipo pilha). É definido por uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. No projeto da nossa equipe, configuramos o autômato para aceitar as cadeias *exclusivamente por pilha vazia*, o que significa que o conjunto de estados finais é considerado vazio ($F = \emptyset$). **Exemplo de Cálculo ($L_2 = a^n b^n$):** Ao receber a palavra $w = aabb$, a máquina lê os 'a's iniciais e empilha o símbolo 'A' duas vezes. Através de uma transição vazia (ou λ -movimento), ela passa a ler os 'b's e, para cada 'b' processado, remove um 'A' da pilha. Ao final da fita de entrada, resta apenas o marcador Z_0 , que também é removido, deixando a pilha vazia e aceitando a palavra.

2.3 Máquina de Turing (MT)

A Máquina de Turing é o modelo de computação mais poderoso, capaz de reconhecer Linguagens Recursivamente Enumeráveis através do uso de uma fita infinita. Ela é definida formalmente por $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$. **Exemplo de Cálculo ($L_4 = a^n b^n c^n$):** Ao processar a cadeia abc , a MT lê o símbolo 'a', escreve 'X' no seu lugar e move o cabeçote para a direita (R) até encontrar um 'b', substituindo-o por 'Y'. Em seguida, move-se novamente para a direita até achar um 'c', mudando-o para 'Z'. Logo após, o cabeçote retrocede para a esquerda (L) para reiniciar a marcação do próximo trio. Se no fim do processo houver apenas símbolos de espaço em branco (B), a computação aceita a palavra.

3 Descrição da Implementação

O projeto foi totalmente codificado em .NET 6+ (C#), focando em traduzir os conceitos matemáticos abstratos para estruturas de dados limpas e eficientes.

3.1 Implementação do AFD

Estruturamos o nosso simulador de AFD utilizando a coleção nativa `HashSet<T>` do C# para representar os conjuntos matemáticos Q, Σ e F . Essa escolha garantiu que os elementos fossem únicos e que as buscas ocorressem em tempo constante $O(1)$. A função de transição δ foi modelada por meio de um `Dictionary<(string, char), string>`, mapeando perfeitamente a relação entre um estado e um caractere lido para o próximo estado. Além da validação da linguagem padrão L_1 , desenvolvemos o parser para ler e montar um AFD genérico a partir de um arquivo JSON externo usando a biblioteca `System.Text.Json`.

3.2 Implementação do Autômato de Pilha

Para construir o simulador de AP, a fita de memória auxiliar foi gerenciada estritamente pela classe genérica `Stack<char>`. Como o desafio dos Palíndromos (L_3) exigia um comportamento não-determinístico, adaptamos a função de transição para retornar uma lista (`List`) de caminhos possíveis. O código gerencia essas ramificações através de uma simulação recursiva de estados, avaliando todas as possibilidades até encontrar um caminho de aceitação.

3.3 Implementação da Máquina de Turing

A principal decisão de projeto da nossa equipe ocorreu na fita da Máquina de Turing. Em vez de utilizarmos estruturas rígidas como *Arrays* ou *Lists*, que exigiriam rotinas pesadas de redimensionamento, optamos por usar um `Dictionary<int, char>`. Nessa estrutura, a chave representa a posição numérica na fita (permitindo índices negativos infinitos) e o valor armazena

o caractere gravado. Essa lógica evitou estouros de memória e nos permitiu simular uma fita bidirecional infinita de forma muito eficiente.

4 Discussão das Questões Reflexivas

Nesta seção, apresentamos as análises conceituais e práticas desenvolvidas pelo grupo ao longo da construção dos simuladores, respondendo às questões propostas para cada modelo de máquina.

4.1 Parte 1: Autômato Finito Determinístico (AFD)

1. Por que a linguagem L_1 é regular? Apresente a expressão regular equivalente.

Resposta: A linguagem L_1 é regular porque se encaixa perfeitamente na definição básica da Teoria da Computação: qualquer linguagem que possa ser reconhecida por um autômato que possui um número finito de estados é, por consequência, uma linguagem regular. Ao criarmos um AFD funcional que processa e aceita apenas as strings que terminam com o sufixo "ab", nós provamos empiricamente a regularidade dessa linguagem. Além disso, de acordo com o Teorema de Kleene, toda linguagem regular pode ser expressa por meio de uma Expressão Regular (ER). No caso da linguagem L_1 , a expressão regular equivalente e simplificada é dada por $r = (a + b)^*(ab)$. Essa notação deixa claro que a cadeia de caracteres pode começar com qualquer sequência ou mistura de letras 'a' e 'b' (representada pelo fecho de Kleene asterisco), incluindo a opção de começar vazia, mas ela deve, obrigatoriamente, terminar com a leitura consecutiva dos símbolos 'a' e 'b'.

2. Descreva formalmente a 5-tupla do AFD construído, justificando a escolha de cada estado.

Resposta: O AFD que desenvolvemos em nosso código pode ser descrito matematicamente pela 5-tupla clássica $M = (Q, \Sigma, \delta, q_0, F)$. Mapeando as variáveis para as estruturas do C#, definimos o conjunto de estados como $Q = \{q_0, q_1, q_2\}$, o alfabeto de símbolos permitidos como $\Sigma = \{a, b\}$, o estado inicial como $q_0 = q_0$ e o conjunto de estados finais contendo apenas $F = \{q_2\}$. A escolha de cada um desses três estados teve uma justificativa lógica baseada no sufixo "ab" que precisávamos reconhecer. O estado q_0 funciona como o ponto de partida e de repouso da máquina, indicando que o autômato ainda não leu o início da sequência final esperada. O estado q_1 é ativado assim que o simulador lê a letra 'a', sinalizando que metade do padrão foi encontrada. Por fim, o estado q_2 representa o sucesso da computação, sendo atingido apenas quando uma letra 'b' é processada logo após um 'a'. Se a máquina ler mais símbolos estando em q_2 , ela recua para q_1 ou retorna para q_0 dependendo do caractere, reiniciando a contagem de forma determinística.

3. O que aconteceria se a função δ não fosse total? Como você tratou entradas inválidas?

das (símbolos fora do alfabeto)?

Resposta: Na teoria, se a função de transição δ de um AFD não for total, significa que existem estados que não possuem regras mapeadas para todos os símbolos do alfabeto. Se a máquina encontrasse uma dessas combinações ausentes, a computação travaria por falta de instruções, quebrando o princípio do determinismo. No desenvolvimento do nosso simulador, lidamos com a não-totalidade da função de transição de um jeito inteligente usando os recursos da linguagem C#. Como salvamos a função δ dentro de um dicionário, utilizamos o método TryGetValue. Se o autômato tentar fazer uma transição que não foi cadastrada, o método retorna falso, e o código interpreta isso como um desvio para um "estado de poço" implícito, rejeitando a palavra na hora. Para tratar símbolos que estão completamente fora do alfabeto Σ , inserimos uma trava de validação logo no início do laço de repetição. Se o programa detectar qualquer caractere inválido na string, o processamento é interrompido imediatamente por segurança, impedindo erros de execução.

4.2 Parte 2: Autômato de Pilha (AP)

1. Por que L_2 não pode ser reconhecida por um AFD? Demonstre usando o Lema do Bombeamento para linguagens regulares.

Resposta: A linguagem $L_2 = \{a^n b^n \mid n \geq 1\}$ não pode ser reconhecida por um AFD por causa de uma limitação física do próprio modelo: autômatos finitos não possuem uma memória auxiliar para guardar dados de longo prazo. Sem uma memória como a pilha, o AFD é incapaz de "contar" quantos 'a's entraram para depois verificar se entrou exatamente a mesma quantidade de 'b's quando a string cresce de forma arbitrária. Conseguimos provar essa limitação usando o Lema do Bombeamento. Vamos supor, por absurdo, que L_2 fosse uma linguagem regular com uma constante de bombeamento p . Se escolhermos uma palavra válida bem longa, como $w = a^p b^p$, o lema nos diz que podemos dividi-la em três partes, $w = xyz$, respeitando as regras de que o tamanho de xy deve ser menor ou igual a p e o tamanho de y deve ser maior que zero. Isso significa que o trecho y (que é a parte que será bombeada) é composto apenas por letras 'a'. Se fizermos o bombeamento duplicando essa parte (gerando xy^2z), vamos aumentar a quantidade de 'a's, mas a quantidade de 'b's vai continuar sendo p . Isso gera uma palavra totalmente desequilibrada que não pertence à linguagem. Como isso quebra a regra do lema, fica provado matematicamente que L_2 não pode ser regular.

2. A aceitação por pilha vazia e por estado final são equivalentes em poder de reconhecimento? Demonstre conceitualmente.

Resposta: Sim, os dois critérios de aceitação — por pilha vazia ou por atingimento de estado final — são totalmente equivalentes em poder computacional. Isso significa que qualquer linguagem livre de contexto que possa ser reconhecida por um método também pode ser reconhecida pelo outro. Podemos demonstrar isso pensando na lógica de conversão entre os modelos. Se tivermos um autômato que aceita cadeias por estado final e quisermos transformá-lo em um

modelo que aceita por pilha vazia, basta criarmos transições vazias (λ -movimentos) partindo de todos os estados finais originais. Essas transições iriam direcionar o fluxo da máquina para um novo estado especial, cujo único objetivo seria rodar um laço para limpar e consumir todos os elementos restantes da pilha. Quando a pilha ficasse totalmente vazia, a palavra seria aceita da mesma forma. O processo inverso também funciona criando um estado final que é atingido quando o fundo da pilha é detectado, provando que a escolha do critério é apenas uma decisão de projeto e não uma limitação teórica.

3. Explique o papel do símbolo Z_0 na sua implementação.

Resposta: O símbolo Z_0 teve um papel fundamental no nosso projeto, servindo como uma solução tanto teórica quanto prática para o funcionamento do autômato de pilha. Na teoria dos autômatos, o Z_0 funciona como um marcador de fundo de pilha. Ele garante que a memória nunca comece totalmente vazia, servindo como um sinalizador para a máquina saber que chegou ao fim dos dados guardados. Já na parte prática da programação em C#, o uso do Z_0 foi uma escolha essencial de engenharia de software para evitar falhas no sistema. Se tentarmos ler ou remover um item de uma estrutura Stack nativa do C# quando ela está vazia, o programa sofre um travamento gerando uma exceção de erro crítico. Colocando o Z_0 como o primeiro elemento da pilha logo na inicialização da máquina, nós criamos um chão seguro para o código trabalhar. No fim do processamento da string, se o cabeçote ler a fita toda e o único caractere que sobrar na pilha for o Z_0 , o simulador tem a certeza de que todos os pares de letras foram casados corretamente. O código então consome o Z_0 por meio de uma transição vazia, esvaziando a pilha de verdade para aceitar a palavra com segurança.

4.3 Parte 3: Máquina de Turing (MT)

1. Por que L_4 não pode ser reconhecida por um Autômato de Pilha? Qual propriedade da MT possibilita o seu reconhecimento?

Resposta: A linguagem $L_4 = \{a^n b^n c^n \mid n \geq 1\}$ pertence à classe das linguagens sensíveis ao contexto e ultrapassa os limites de um autômato de pilha comum. O problema de usar uma pilha aqui é que o acesso à memória é muito restrito: você só pode interagir com o elemento que está no topo (padrão LIFO). O autômato até consegue empilhar os 'a's e ir removendo-os conforme lê os 'b's para garantir que as quantidades batem. Mas, assim que a leitura dos 'b's termina, a pilha fica vazia e a máquina perde a informação de qual era o tamanho de n . Com isso, ela perde a capacidade de validar se a quantidade de 'c's que vem a seguir também é igual. A Máquina de Turing consegue resolver essa limitação graças a duas propriedades exclusivas do seu modelo: ela possui uma fita de leitura e escrita ilimitada e um cabeçote que se move em duas direções (para a esquerda e para a direita). Essas características dão à máquina um acesso livre à memória, permitindo que ela navegue em zigue-zague pela fita, indo e voltando quantas vezes forem necessárias para conferir e marcar os símbolos sem apagar o histórico anterior.

2. Quantos estados foram necessários para a MT de L_4 ? Descreva a estratégia de “marcação” adotada.

Resposta: Para construir o reconhecedor da linguagem L_4 em nosso simulador, mapeamos a função de transição utilizando exatamente seis estados comportamentais: ‘q0, q1, q2, q3, q4’ e o estado final de aceitação ‘q_{acc}’. A estratégia que adotamos para fazer a máquina funcionar baseou-se em uma rotina de marcação e varredura passo a passo. A computação começa no estado ‘q0’, que procura o primeiro ‘o’ para ‘Y’ e passando para o estado ‘q2’. O estado ‘q2’ continua avançando para a direita, pulando os ‘b’s até achar o ‘o’ para ‘Z’. Assim que o trio é formado, o autômato entra no estado ‘q3’, que funciona com uma rotina para rebobinar.

3. Um computador moderno é mais poderoso que uma Máquina de Turing? Justifique à luz da Tese de Church-Turing.

Resposta: Quando analisamos a questão pelo lado da teoria computacional puro, a resposta é não: um computador moderno não é mais poderoso do que uma Máquina de Turing. A famosa Tese de Church-Turing postula que qualquer processo ou cálculo matemático que possa ser resolvido de forma mecânica através de um algoritmo pode ser inteiramente executado por uma Máquina de Turing. Os computadores modernos que usamos no nosso dia a dia nada mais são do que implementações físicas, altamente otimizadas e muito rápidas desse mesmo conceito abstrato de Turing. Na verdade, se pensarmos estritamente pela teoria, a Máquina de Turing abstrata é até mais poderosa do que um computador real de mercado. Isso acontece porque o modelo de Turing assume uma fita de memória que é matematicamente infinita, enquanto os computadores físicos sempre vão esbarrar nas limitações materiais do mundo real, ficando restritos à capacidade máxima de armazenamento dos seus pentes de memória RAM e dos seus discos rígidos.

5 Resultados e Testes

As saídas no terminal (Console) corroboram a corretude matemática das máquinas construídas. Os rastreamentos instantâneos das memórias (traces) mostraram-se exatos. Abaixo, os resultados processados na íntegra para os arquivos de entrada exigidos:

AFD (Parte 1)

Diagrama de Transições (Tabela)

Estado	a	b	

q0	q1	q0	
q1	q1	q2	
q2	q1	q0	

Processando entradas.txt para L1...

```

Cadeia: ab
Rastro: q0 -> q1 -> q2
Resultado: ACEITA
-----
Cadeia: aab
Rastro: q0 -> q1 -> q1 -> q2
Resultado: ACEITA
-----
Cadeia: bab
Rastro: q0 -> q0 -> q1 -> q2
Resultado: ACEITA
-----
Cadeia: ababab
Rastro: q0 -> q1 -> q2 -> q1 -> q2 -> q1 -> q2
Resultado: ACEITA
-----
Cadeia: ba
Rastro: q0 -> q0 -> q1
Resultado: REJEITA
-----
Cadeia: vazia
Rastro: q0
Resultado: REJEITA
-----
Cadeia: b
Rastro: q0 -> q0
Resultado: REJEITA
-----

```

```

Carregando afd.json
Máquina carregada com sucesso do arquivo JSON!

```

Diagrama de Transições (Tabela)

Estado	0	1
q0	q0	q1
q1	q1	q0

=== Simulador de Autômato de Pilha (Parte 2) ===

=== L2: $a^n b^n$ ($n \geq 1$) ===

```

Processando: ab
Configuração: (q: q0, w: ab, Pilha: Z)
Configuração: (q: q0, w: b, Pilha: AZ)

```


Configuração: (q: q1, w: b, Pilha: AZ)
Configuração: (q: q1, w: E, Pilha: Z)
Configuração: (q: q2, w: E, Pilha: [Vazia])
Resultado: ACEITA

Processando: aabb

Configuração: (q: q0, w: aabb, Pilha: Z)
Configuração: (q: q0, w: abb, Pilha: AZ)
Configuração: (q: q1, w: abb, Pilha: AZ)
Configuração: (q: q0, w: bb, Pilha: AAZ)
Configuração: (q: q1, w: bb, Pilha: AAZ)
Configuração: (q: q1, w: b, Pilha: AZ)
Configuração: (q: q1, w: E, Pilha: Z)
Configuração: (q: q2, w: E, Pilha: [Vazia])
Resultado: ACEITA

Processando: aaabbb

Configuração: (q: q0, w: aaabbb, Pilha: Z)
Configuração: (q: q0, w: aabbb, Pilha: AZ)
Configuração: (q: q1, w: aabbb, Pilha: AZ)
Configuração: (q: q0, w: abbb, Pilha: AAZ)
Configuração: (q: q1, w: abbb, Pilha: AAZ)
Configuração: (q: q0, w: bbb, Pilha: AAAZ)
Configuração: (q: q1, w: bbb, Pilha: AAAZ)
Configuração: (q: q1, w: bb, Pilha: AAZ)
Configuração: (q: q1, w: b, Pilha: AZ)
Configuração: (q: q1, w: E, Pilha: Z)
Configuração: (q: q2, w: E, Pilha: [Vazia])
Resultado: ACEITA

Processando: aab

Configuração: (q: q0, w: aab, Pilha: Z)
Configuração: (q: q0, w: ab, Pilha: AZ)
Configuração: (q: q1, w: ab, Pilha: AZ)
Configuração: (q: q0, w: b, Pilha: AAZ)
Configuração: (q: q1, w: b, Pilha: AAZ)
Configuração: (q: q1, w: E, Pilha: AZ)
Resultado: REJEITA

Processando: abb

Configuração: (q: q0, w: abb, Pilha: Z)
Configuração: (q: q0, w: bb, Pilha: AZ)
Configuração: (q: q1, w: bb, Pilha: AZ)

Configuração: (q: q1, w: b, Pilha: Z)
Configuração: (q: q2, w: b, Pilha: [Vazia])
Resultado: REJEITA

Processando: ba
Configuração: (q: q0, w: ba, Pilha: Z)
Resultado: REJEITA

Processando: vazia
Configuração: (q: q0, w: E, Pilha: Z)
Resultado: REJEITA

Processando: abab
Configuração: (q: q0, w: abab, Pilha: Z)
Configuração: (q: q0, w: bab, Pilha: AZ)
Configuração: (q: q1, w: bab, Pilha: AZ)
Configuração: (q: q1, w: ab, Pilha: Z)
Configuração: (q: q2, w: ab, Pilha: [Vazia])
Resultado: REJEITA

=== Desafio L3: Palíndromos sobre {a,b} ===

Processando: a
Configuração: (q: q0, w: a, Pilha: Z)
Configuração: (q: q1, w: a, Pilha: Z)
Configuração: (q: q2, w: a, Pilha: [Vazia])
Configuração: (q: q0, w: E, Pilha: AZ)
Configuração: (q: q1, w: E, Pilha: AZ)
Configuração: (q: q1, w: E, Pilha: Z)
Configuração: (q: q2, w: E, Pilha: [Vazia])
Resultado: ACEITA

Processando: aba
Configuração: (q: q0, w: aba, Pilha: Z)
Configuração: (q: q1, w: aba, Pilha: Z)
Configuração: (q: q2, w: aba, Pilha: [Vazia])
Configuração: (q: q0, w: ba, Pilha: AZ)
Configuração: (q: q1, w: ba, Pilha: AZ)
Configuração: (q: q0, w: a, Pilha: BAZ)
Configuração: (q: q1, w: a, Pilha: BAZ)
Configuração: (q: q0, w: E, Pilha: ABAZ)
Configuração: (q: q1, w: E, Pilha: ABAZ)

Configuração: (q: q1, w: E, Pilha: BAZ)
 Configuração: (q: q1, w: a, Pilha: AZ)
 Configuração: (q: q1, w: E, Pilha: Z)
 Configuração: (q: q2, w: E, Pilha: [Vazia])
 Configuração: (q: q1, w: ba, Pilha: Z)
 Configuração: (q: q2, w: ba, Pilha: [Vazia])
 Resultado: ACEITA

Processando: abba

Configuração: (q: q0, w: abba, Pilha: Z)
 Configuração: (q: q1, w: abba, Pilha: Z)
 Configuração: (q: q2, w: abba, Pilha: [Vazia])
 Configuração: (q: q0, w: bba, Pilha: AZ)
 Configuração: (q: q1, w: bba, Pilha: AZ)
 Configuração: (q: q0, w: ba, Pilha: BAZ)
 Configuração: (q: q1, w: ba, Pilha: BAZ)
 Configuração: (q: q1, w: a, Pilha: AZ)
 Configuração: (q: q1, w: E, Pilha: Z)
 Configuração: (q: q2, w: E, Pilha: [Vazia])
 Configuração: (q: q1, w: ba, Pilha: AZ)
 Configuração: (q: q1, w: bba, Pilha: Z)
 Configuração: (q: q2, w: bba, Pilha: [Vazia])
 Resultado: ACEITA

Processando: ab

Configuração: (q: q0, w: ab, Pilha: Z)
 Configuração: (q: q1, w: ab, Pilha: Z)
 Configuração: (q: q2, w: ab, Pilha: [Vazia])
 Configuração: (q: q0, w: b, Pilha: AZ)
 Configuração: (q: q1, w: b, Pilha: AZ)
 Configuração: (q: q0, w: E, Pilha: BAZ)
 Configuração: (q: q1, w: E, Pilha: BAZ)
 Configuração: (q: q1, w: E, Pilha: AZ)
 Configuração: (q: q1, w: b, Pilha: Z)
 Configuração: (q: q2, w: b, Pilha: [Vazia])
 Resultado: REJEITA

Processando: aab

Configuração: (q: q0, w: aab, Pilha: Z)
 Configuração: (q: q1, w: aab, Pilha: Z)
 Configuração: (q: q2, w: aab, Pilha: [Vazia])
 Configuração: (q: q0, w: ab, Pilha: AZ)
 Configuração: (q: q1, w: ab, Pilha: AZ)
 Configuração: (q: q1, w: b, Pilha: Z)

```

Configuração: (q: q2, w: b, Pilha: [Vazia])
Configuração: (q: q0, w: b, Pilha: AAZ)
Configuração: (q: q1, w: b, Pilha: AAZ)
Configuração: (q: q0, w: E, Pilha: BAAZ)
Configuração: (q: q1, w: E, Pilha: BAAZ)
Configuração: (q: q1, w: E, Pilha: AAZ)
Configuração: (q: q1, w: b, Pilha: AZ)
Configuração: (q: q1, w: ab, Pilha: Z)
Configuração: (q: q2, w: ab, Pilha: [Vazia])
Resultado: REJEITA
-----

```

=== Simulador de Máquina de Turing (Parte 3) ===

=== L4: $a^n b^n c^n$ ($n \geq 1$) ===

```

Processando L4: abc
Fita: ... _ [q0>a] b c ...
Fita: ... X [q1>b] c ...
Fita: ... X Y [q2>c] _ ...
Fita: ... X [q3>Y] Z ...
Fita: ... _ [q3>X] Y Z ...
Fita: ... X [q0>Y] Z ...
Fita: ... X Y [q4>Z] _ ...
Fita: ... X Y Z [q4>_] _ ...
Fita: ... X Y Z _ [q_acc>_] _ ...
Resultado: ACEITA
-----

```

```

Processando L4: aabbcc
Fita: ... _ [q0>a] a b b c c ...
Fita: ... X [q1>a] b b c c ...
Fita: ... X a [q1>b] b c c ...
Fita: ... X a Y [q2>b] c c ...
Fita: ... X a Y b [q2>c] c ...
Fita: ... X a Y [q3>b] Z c ...
Fita: ... X a [q3>Y] b Z c ...
Fita: ... X [q3>a] Y b Z c ...
Fita: ... _ [q3>X] a Y b Z c ...
Fita: ... X [q0>a] Y b Z c ...
Fita: ... X X [q1>Y] b Z c ...
Fita: ... X X Y [q1>b] Z c ...
Fita: ... X X Y Y [q2>Z] c ...
Fita: ... X X Y Y Z [q2>c] _ ...
Fita: ... X X Y Y [q3>Z] Z ...
Fita: ... X X Y [q3>Y] Z Z ...
Fita: ... X X [q3>Y] Y Z Z ...

```

```

Fita: ... X [q3>X] Y Y Z Z ...
Fita: ... X X [q0>Y] Y Z Z ...
Fita: ... X X Y [q4>Y] Z Z ...
Fita: ... X X Y Y [q4>Z] Z ...
Fita: ... X X Y Y Z [q4>Z] _ ...
Fita: ... X X Y Y Z Z [q4>_] _ ...
Fita: ... X X Y Y Z Z _ [q_acc>_] _ ...
Resultado: ACEITA

```

Processando L4: aaabbbccc

```

Fita: ... _ [q0>a] a a b b b c c c ...
Fita: ... X [q1>a] a b b b c c c ...
Fita: ... X a [q1>a] b b b c c c ...
Fita: ... X a a [q1>b] b b c c c ...
Fita: ... X a a Y [q2>b] b c c c ...
Fita: ... X a a Y b [q2>b] c c c ...
Fita: ... X a a Y b b [q2>c] c c ...
Fita: ... X a a Y b [q3>b] Z c c ...
Fita: ... X a a Y [q3>b] b Z c c ...
Fita: ... X a a [q3>Y] b b Z c c ...
Fita: ... X a [q3>a] Y b b Z c c ...
Fita: ... X [q3>a] a Y b b Z c c ...
Fita: ... _ [q3>X] a a Y b b Z c c ...
Fita: ... X [q0>a] a Y b b Z c c ...
Fita: ... X X [q1>a] Y b b Z c c ...
Fita: ... X X a [q1>Y] b b Z c c ...
Fita: ... X X a Y [q1>b] b Z c c ...
Fita: ... X X a Y Y [q2>b] Z c c ...
Fita: ... X X a Y Y b [q2>Z] c c ...
Fita: ... X X a Y Y b Z [q2>c] c ...
Fita: ... X X a Y Y b [q3>Z] Z c ...
Fita: ... X X a Y Y [q3>b] Z Z c ...
Fita: ... X X a Y [q3>Y] b Z Z c ...
Fita: ... X X a [q3>Y] Y b Z Z c ...
Fita: ... X X [q3>a] Y Y b Z Z c ...
Fita: ... X [q3>X] a Y Y b Z Z c ...
Fita: ... X X [q0>a] Y Y b Z Z c ...
Fita: ... X X X [q1>Y] Y b Z Z c ...
Fita: ... X X X Y [q1>Y] b Z Z c ...
Fita: ... X X X Y Y [q1>b] Z Z c ...
Fita: ... X X X Y Y Y [q2>Z] Z c ...
Fita: ... X X X Y Y Y Z [q2>Z] c ...
Fita: ... X X X Y Y Y Z Z [q2>c] _ ...
Fita: ... X X X Y Y Y Z [q3>Z] Z ...
Fita: ... X X X Y Y Y [q3>Z] Z Z ...
Fita: ... X X X Y Y [q3>Y] Z Z Z ...

```

```

Fita: ... X X X Y [q3>Y] Y Z Z Z ...
Fita: ... X X X [q3>Y] Y Y Z Z Z ...
Fita: ... X X [q3>X] Y Y Y Z Z Z ...
Fita: ... X X X [q0>Y] Y Y Z Z Z ...
Fita: ... X X X Y [q4>Y] Y Z Z Z ...
Fita: ... X X X Y Y [q4>Y] Z Z Z ...
Fita: ... X X X Y Y Y [q4>Z] Z Z ...
Fita: ... X X X Y Y Y Z [q4>Z] Z ...
Fita: ... X X X Y Y Y Z Z [q4>Z] _ ...
Fita: ... X X X Y Y Y Z Z Z [q4>_] _ ...
Fita: ... X X X Y Y Y Z Z Z _ [q_acc>_] _ ...
Resultado: ACEITA

```

Processando L4: aabbc

```

Fita: ... _ [q0>a] a b b c ...
Fita: ... X [q1>a] b b c ...
Fita: ... X a [q1>b] b c ...
Fita: ... X a Y [q2>b] c ...
Fita: ... X a Y b [q2>c] _ ...
Fita: ... X a Y [q3>b] Z ...
Fita: ... X a [q3>Y] b Z ...
Fita: ... X [q3>a] Y b Z ...
Fita: ... _ [q3>X] a Y b Z ...
Fita: ... X [q0>a] Y b Z ...
Fita: ... X X [q1>Y] b Z ...
Fita: ... X X Y [q1>b] Z ...
Fita: ... X X Y Y [q2>Z] _ ...
Fita: ... X X Y Y Z [q2>_] _ ...
Resultado: REJEITA

```

Processando L4: ab

```

Fita: ... _ [q0>a] b ...
Fita: ... X [q1>b] _ ...
Fita: ... X Y [q2>_] _ ...
Resultado: REJEITA

```

Processando L4: abc abc

```

Fita: ... _ [q0>a] b c a b c ...
Fita: ... X [q1>b] c a b c ...
Fita: ... X Y [q2>c] a b c ...
Fita: ... X [q3>Y] Z a b c ...
Fita: ... _ [q3>X] Y Z a b c ...
Fita: ... X [q0>Y] Z a b c ...
Fita: ... X Y [q4>Z] a b c ...

```

Fita: ... X Y Z [q4>] a b c ...
 Resultado: REJEITA

Processando L4: vazia

Fita: ... _ [q0>_] _ ...
 Resultado: REJEITA

=== Desafio: Computando $f(n) = n + 1$ (Adição Unária) ===

Processando $f(n)$ para $n = 1$ (Cadeia: 1)

Fita: ... _ [q0>1] _ ...
 Fita: ... 1 [q0>_] _ ...
 Fita: ... 1 1 [q_acc>_] _ ...
 Resultado computado com sucesso!

Processando $f(n)$ para $n = 2$ (Cadeia: 11)

Fita: ... _ [q0>1] 1 ...
 Fita: ... 1 [q0>1] _ ...
 Fita: ... 1 1 [q0>_] _ ...
 Fita: ... 1 1 1 [q_acc>_] _ ...
 Resultado computado com sucesso!

Processando $f(n)$ para $n = 4$ (Cadeia: 1111)

Fita: ... _ [q0>1] 1 1 1 ...
 Fita: ... 1 [q0>1] 1 1 ...
 Fita: ... 1 1 [q0>1] 1 ...
 Fita: ... 1 1 1 [q0>1] _ ...
 Fita: ... 1 1 1 1 [q0>_] _ ...
 Fita: ... 1 1 1 1 1 [q_acc>_] _ ...
 Resultado computado com sucesso!

Tabela 1: Resultados dos Testes Processados no Sistema

Máquina	Linguagem	Cadeia Processada	Resultado Sistema
AFD	L_1 (Termina em ab)	$ababab$	ACEITA
AFD	L_1 (Termina em ab)	ba	REJEITA
AP	$L_2 = a^n b^n$	$aabb$	ACEITA
AP	$L_2 = a^n b^n$	abb	REJEITA
AP	L_3 (Palíndromo)	$abba$	ACEITA
MT	$L_4 = a^n b^n c^n$	$aaabbbccc$	ACEITA
MT	Adição Unária $f(n)$	$1111 \rightarrow 11111$	ACEITA (Parada)

6 Análise Comparativa de Poder Computacional

O desenvolvimento consecutivo destas três máquinas permitiu uma análise clara e prática da Hierarquia de Chomsky.

Os **Autômatos Finitos** (AFDs) demonstraram possuir o poder computacional mais restrito. Por não possuírem memória auxiliar, sua aplicação limita-se a reconhecer padrões sequenciais e repetições (como L_1). Eles não conseguem, por exemplo, contar ou balancear símbolos. A tentativa de processar $a^n b^n$ com um AFD exigiria um número infinito de estados, o que viola a sua definição.

Os **Autômatos de Pilha** (APs) resolvem o problema da contagem ao adicionar a estrutura de pilha (memória finita em profundidade). Com isso, tornam-se capazes de reconhecer pares balanceados, como casar os 'a's com os 'b's (L_2). Contudo, a limitação de acesso estrito ao topo da pilha impede que o AP reconheça padrões complexos de cruzamento tridimensional, tornando-o incapaz de lidar com linguagens como $a^n b^n c^n$.

As **Máquinas de Turing** (MT) demonstraram o ápice do poder computacional. Com a introdução de uma fita de leitura/escrita infinita e um cabeçote móvel para a esquerda e direita, a MT atinge o conceito de completude de Turing. Esta máquina não apenas consegue memorizar e contar, mas também remarcar dados lidos anteriormente (essencial para reconhecer L_4) e realizar cálculos numéricos absolutos, como comprovado pela execução da máquina de adição unária.

7 Conclusão

A execução deste projeto proporcionou uma solidificação inestimável dos conceitos abordados na disciplina. Traduzir o jargão matemático rigoroso (como 5-tuplas e relações de transição) para código limpo e funcional em C# quebrou a abstração teórica, provando que matrizes, dicionários e pilhas do cotidiano de desenvolvimento de software são, no fundo, modelos matemáticos.

Compreender o porquê de um JSON Parser ou Compilador precisar de uma árvore estruturada em vez de um simples RegEx (pois RegEx não tem memória de pilha) muda a forma de arquitetar softwares no mercado de trabalho. A teoria da computação deixa de ser uma disciplina isolada e consolida-se como a base irrefutável de qualquer processo de engenharia de software moderno.

Referências

- [1] MENEZES, P. B. *Linguagens formais e autômatos*. 6. ed. Porto Alegre: Bookman, 2010.

- [2] HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introdução à teoria de autômatos, linguagens e computação*. Rio de Janeiro: Elsevier, 2002.
- [3] SIPSER, M. *Introdução à teoria da computação*. São Paulo: Cengage Learning, 2007.